



Iowa State University
COM S 459

S3 Secure File System

By: Evan Litzer and Anvitha Sri

Problem Scenario

A medical office uploads their patient records to cloud storage.

- Misconfigured storage bucket or employee credentials are stolen
- Every patient file exposed in plaintext
- Cloud provider could read them at any time! Office had no way to prevent it

This actually happened in 2023 → MOVEit Data Breach

- 1000+ organizations and 95 millions individuals affected
- Files were stored and transferred with no client-side encryption
- Access control did not work, every storable file was readable

If cloud provider stores your files, can you really trust that only you can read them?

Problem:

Cloud Storage stores files in plaintext

- Cloud providers like AWS S3 have access control.
 - If access is breached, then every file is exposed in readable plaintext
 - SSE is handled by cloud provider → provider holds keys and can decrypt data
- No mechanism for users to control who read files at cryptographic level

System Assumptions:

- Cloud provider is untrusted, user device is trusted
- Authentication is not strong enough, as stolen credentials can expose everything

Implementation Goals:

1. Files must be encrypted before they enter cloud
2. Only the recipient of a file can decrypt
3. Access control is enforced at cryptographic and account level
4. Sharing must be per file and per user.

	Normal Cloud Storage	S3 Secure Storage
Who holds keys?	Cloud provider	User
Breach exposes plaintext?	Yes	No (ciphertext)
Sharing Level	Bucket/Folder level	Per file and user
Provider can read files?	Yes	No

Conceptual Design

React Frontend: User interface – only point of user interaction

- (file upload/download/sharing/login+signup)

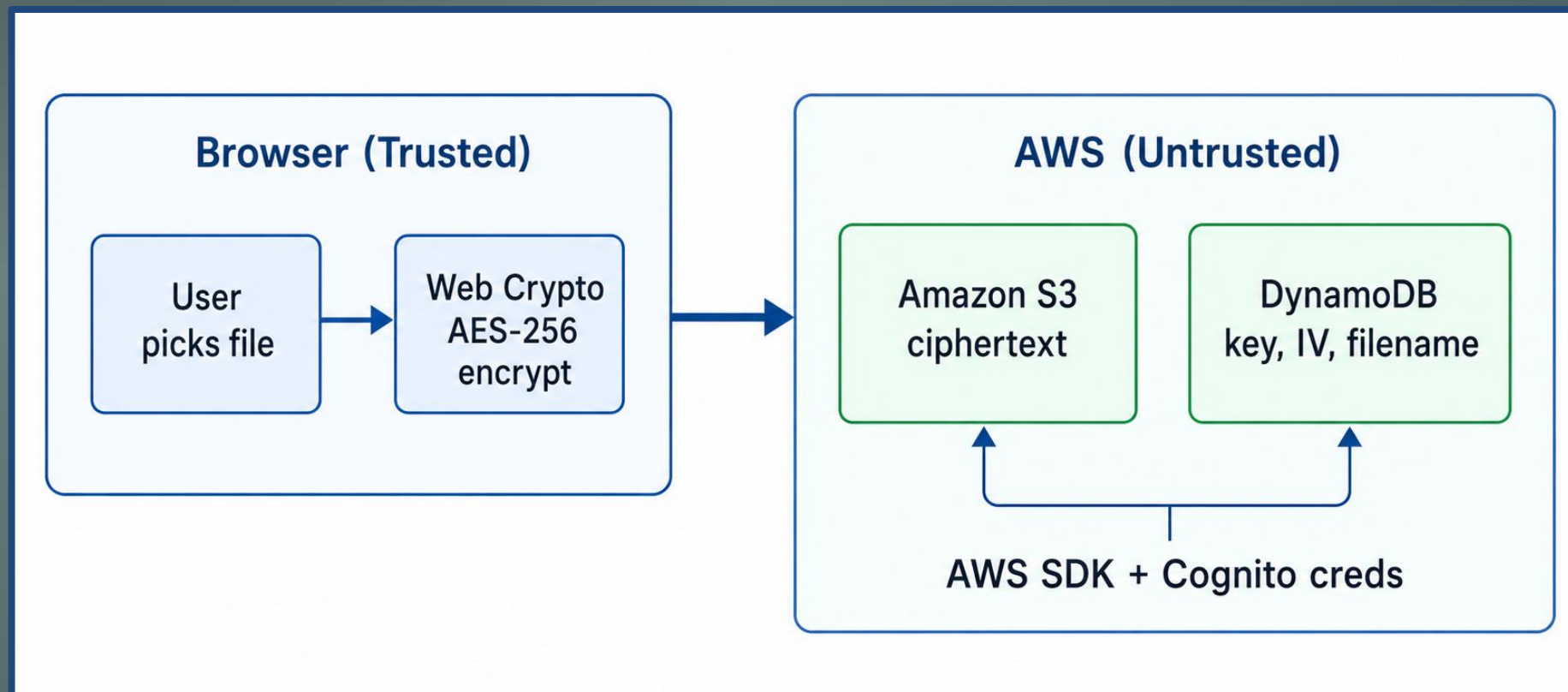
Web Crypto API: In-browser encryption/decryption engine

AWS Cognito: Identity and Authentication

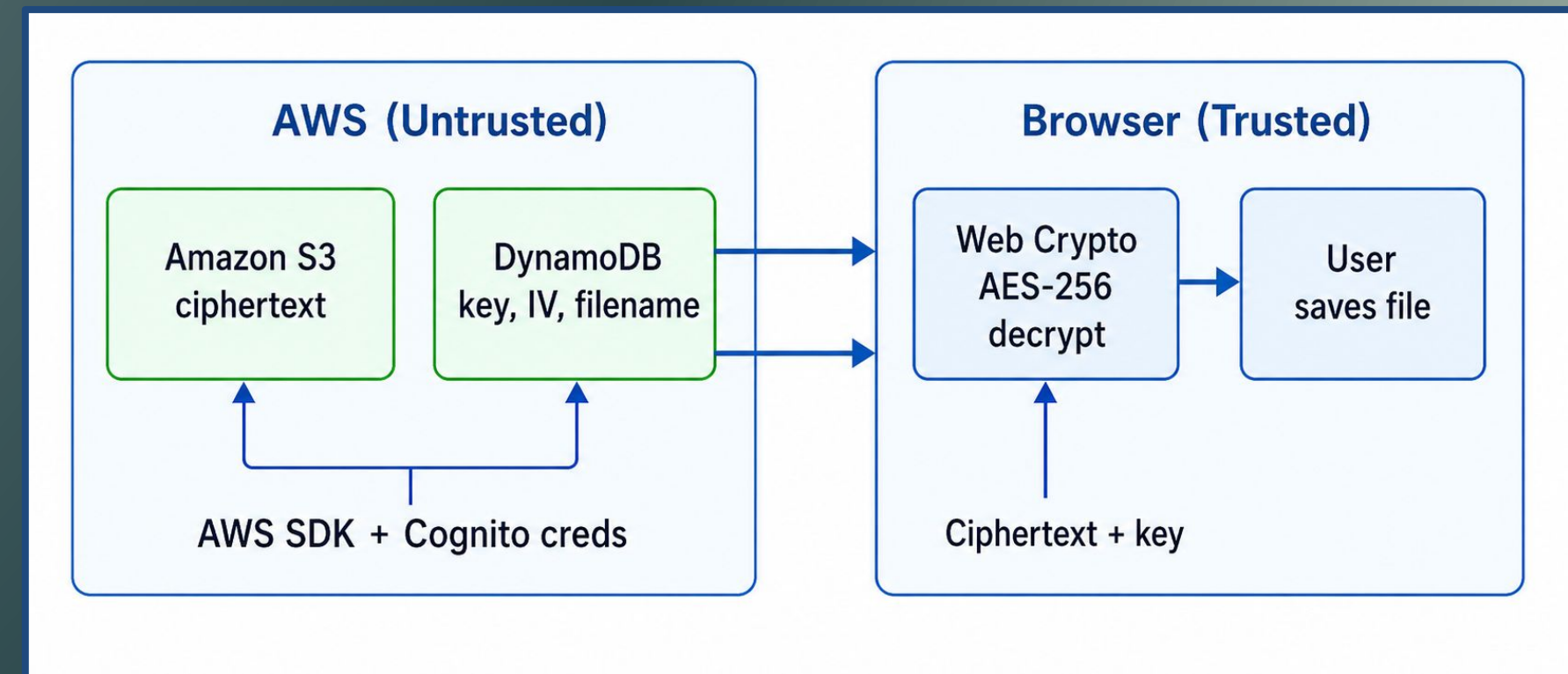
Amazon S3: Encrypted file storage

Amazon DynamoDB: Key storage, file metadata, sharing records

Upload Flow:



Download Flow:



Core Design Principles:

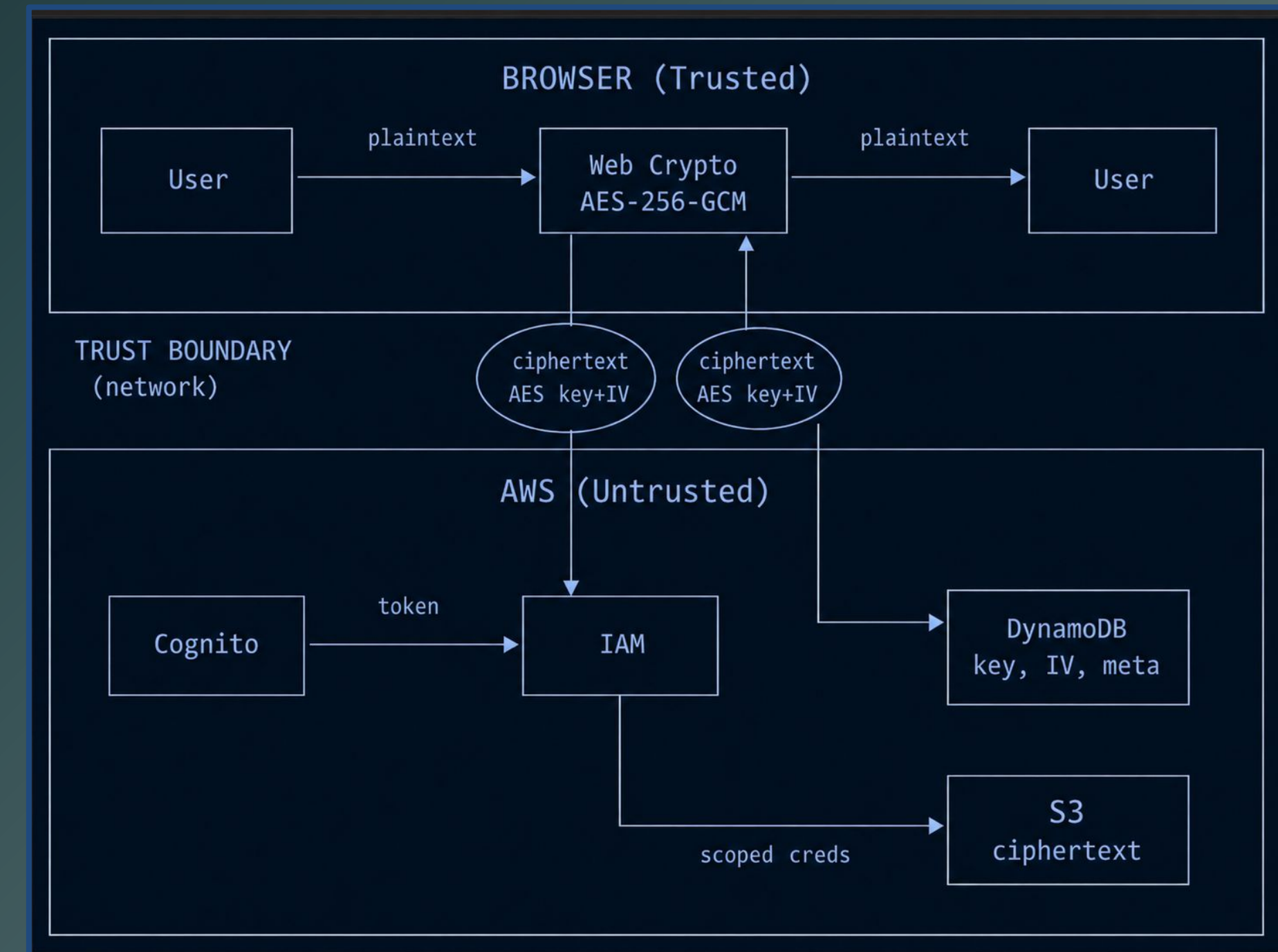
- **Encrypt first:**
 - No plaintext ever reaches AWS → no provider trust
- **Per File Keys:**
 - Each file has its own AES-256-GCM key
- **Credential Scoping:**
 - Cognito Identity Pool issues temporary credentials
 - IAM restricts to S3 and DynamoDB
- **Application Level Sharing:**
 - Sharing is enforced by application.
 - Recipient identity ID must appear in Shared Files table before app fetches key for them.

Trusted Zone (Browser):

- Plaintext file
- Key generation
- Encryption/Decryption
- Nothing leaves unencrypted.

Untrusted Zone (AWS):

- S3 receives and stores ciphertext
- DynamoDB stores keys
- Cognito manages identity



Implementation

Layer	Tool/Service
Frontend	React 19 and Vite
Authorization UI	@aws-amplify/ui-react
AWS SDK	@aws-sdk/client-s3, @aws-sdk/client-dynamo db
Encryption	Web Crypto API in browser
Routing	React
Cloud	AWS (Cognito, S3, DynamoDB, IAM)

FileMetadata

userId: owner's Cognito identity ID
s3Key: timestamp and filename
filename: original filename
exportedKey: base64 AES-256 key
iv: base64 12 byte IV

One record
per uploaded
file.

SharedFiles

recipientId: recipient's Cognito identity ID
s3Key: identifies the file
ownerUserId: used to build s3 path
exportedKey: base64 AES key
iv: base64 IV

One record
per sharing
relationship.

Users

email: user's email (partition key)
userId: Cognito identity ID

One record
per registered
user.

Implementation

Encryption:

1. **Key generation:** `crypto.subtle.generateKey` — AES-GCM, 256-bit, extractable
2. **Encryption:** `crypto.subtle.encrypt` — AES-GCM with random 12-byte IV
3. **Decryption:** `crypto.subtle.decrypt` — same key and IV, returns plaintext

Key is exported as raw bytes and base64-encoded for storage in DynamoDB. When downloaded, it is re-imported before decryption

Encryption Layer:

Handles all cryptographic operations in browser with Web Crypto API

Storage Layer:

Communicates with S3 and DynamoDB from browser using temp AWS credentials issued by Cognito

Sharing Layer:

Manages user registration and file sharing relationships through DynamoDB. Access control at app level

User Guide

01 Sign Up / Log In

Go to the app URL and create an account with your email and password. Cognito handles authentication — no separate login required.

02 Upload a File

Click Upload and select any file from your device. The file is AES-256-GCM encrypted in your browser before it ever reaches AWS. It appears in Your Files once uploaded.

03 Download a File

Click Download next to any file in Your Files. The app fetches the ciphertext from S3 and the key from DynamoDB, decrypts it in-browser, and saves the original file.

04 Share a File

Enter the recipient's email in the Share field. A new entry is created in SharedFiles — the recipient can now see and download that file from their Shared with Me tab.

Evaluation

Did we actually accomplish our design goals? Yes — here's the evidence.

✓ **Files encrypted before entering cloud**

Web Crypto API's AES-256-GCM runs entirely in the browser. The S3 PutObject call only ever receives ciphertext — AWS never sees plaintext.

✓ **Only the recipient can decrypt**

Each file has a unique AES key stored in DynamoDB. On download, the key is fetched and re-imported into the browser before decryption. No key = no access.

✓ **Access control at cryptographic + account level**

Cognito Identity Pool issues scoped temporary credentials via IAM. Users can only access paths under their own identity ID in S3 and DynamoDB.

✓ **Per-file, per-user sharing**

Sharing writes a new record to SharedFiles with the recipient's Cognito identity ID and a copy of the key + IV. Access is checked at the application level before any key is fetched.

Demo

Live walkthrough

01 Sign Up / Log In

Create a new account with email + password. Cognito handles verification.

02 Upload a File

Pick any file → click Encrypt & Upload. Watch the status: Encrypting → Uploading → Done.

03 Download the File

Click Download. File decrypts in browser and saves to disk — original file, intact.

04 Share with Another User

Enter a second account email → Share. Log in as recipient → see file in Shared with You → Download.